Università degli Studi di Milano

Dipartimento di Informatica Giovanni Degli Antoni

Statistical Methods for Machine Learning: Surrogate Losses and Classification

Stefano Capelli

June 27, 2026

Abstract

This project aims to study the classification challenge of minimizing the 0-1 loss, replacing it with a surrogate loss, offering an experimental view on theoretical papers like [1]. The surrogate losses used in this study are the hinge loss, the logistic loss and the squared loss, analyzing how well each of these acts as a proxy for classification error.

1 Surrogate Losses and CLassification

1.1 Theoretical Background

A central challenge in binary classification is the direct optimization of the true classification error, formalized by the 0-1 loss function:

$$\ell_{0-1}(Y, f(X)) = I(Y \cdot f(x) < 0)$$

Due to the nature of the 0-1 loss as discontinuous and non-convex with respect to its margin, direct minimization of the empirical risk is NP-hard. In order to bypass this computational intractability, the theory relies on minimizing empirically tractable, convex surrogate loss functions $\phi(Yf(X))$. While choosing a convex surrogate ensures computational efficiency, [1] the article states that a surrogate loss function is valid if it is classification-calibrated as a necessary and sufficient condition to upper bound the true risk to the surrogate risk and so if minimizing its expected risk implies minimizing the classification error. In order to be classification-calibrated, a convex loss function needs to satisfy the condition that the derivative of the loss at zero must be strictly negative.

1.2 Surrogate Losses

In this project, three surrogate losses that meet these requirements are implemented and tested on the data:

1. **Hinge Loss**: $\phi(\alpha) = \max(0, 1 - \alpha)$
2. **Logistic Loss**: $\phi(\alpha) = \ln(1 + e^{-2\alpha})$
3. **Squared Loss**: $\phi(\alpha) = (1 - \alpha)^2$

1.3 The Algorithm

To minimize this Regularized Empirical Risk across our datasets, we implement a unified optimization engine formulated as:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \left(\frac{1}{n} \sum_{i=1}^{n} \phi(y_i \mathbf{w}^\top \mathbf{x}_i) + \frac{\lambda}{2} \|\mathbf{w}_{1:}\|_2^2 \right) \tag{1}$$

where $\frac{\lambda}{2}\|\mathbf{w}_{1:}\|_2^2$ is an L_2 regularization penalty (excluding the bias w_0) introduced to control model complexity and prevent overfitting.

Stochastic Gradient Descent (SGD) is used to solve this objective.

Algorithm 1 Stochastic Gradient Descent

Parameters: number of T of rounds, learning rates $\eta_1, \ldots, \eta_T > 0$
Input Training set (x_1, y_1), ..., $(x_m, y_m) \in R^d \times \{-1, 1\}$
Set $w_1 = 0$
for $t = 1, \ldots, T$ **do**

1. Draw uniformly at random an index $Zt \in 1, \ldots, m$ of a training example and obtain the corresponding loss ℓ_{Z_t}
2. Perform a gradient step $w_{t+1} = w_t - \eta_t \nabla \ell_{Z_t}(w_t)$

Output: $\overline{w} = \frac{1}{T}(w_1 + \cdots + w_T)$

In this specific setting, the gradient step also involves the regularization term to enforce stability and the weight vector skips the first item that is dedicated to the bias. The learning rate formula is kept standard for all losses and does not use the strong convexity rule in order to have only the loss function as changing factors and to be able to have a better evaluation on the differences.

2 Datasets

2.1 Synthetic datasets

In order to produce synthetic data the main approaches evaluated in this case of study has been:

- Generate uniformly distributed data within a square in the positive and negative sides of the axes, draw a line passing through the origin and set the positive side points as +1 and the ones on the negative side as -1
- Generate clusters of points using a Gaussian Distribution, using mean and covariance to set the starting point and the shape. +1 and -1 are set based on the position of the starting point in the positive or negative side.

Both of these techniques can be used for the linearly separable and the non-separable classification datasets, in particular, changing or adding some steps. Using uniformly distributed data, in order to make it non-separable, some randomly extracted points are changed (from -1 to +1 and vice-versa), artificially adding noise to the distribution. In the Gaussian generation, simply moving the centers of the data closer to each other is enough to generate noisy data that is positioned inside the opposite class region. In this project, the Gaussian has been identified as the better solution and generally more accurate as a simulation, it has been kept as bi-dimensional for better understanding and plotting.

2.2 Real-world dataset

In order to experiment on real-world data, the Iris Dataset (from *sklearn.datasets*) has been selected and utilized after applying some changes to its structure. The first transformation is to reduce the number of classes to 2 limiting the possible classes to *Iris Setosa* and *Iris Versicolor*, and in order to compare it to the synthetic data, the dimensions have been reduced to 2, only utilizing sepal length and width in a bidimensional space. Originally the classes are 0 and 1, so a renaming in +1 and -1 has been necessary.

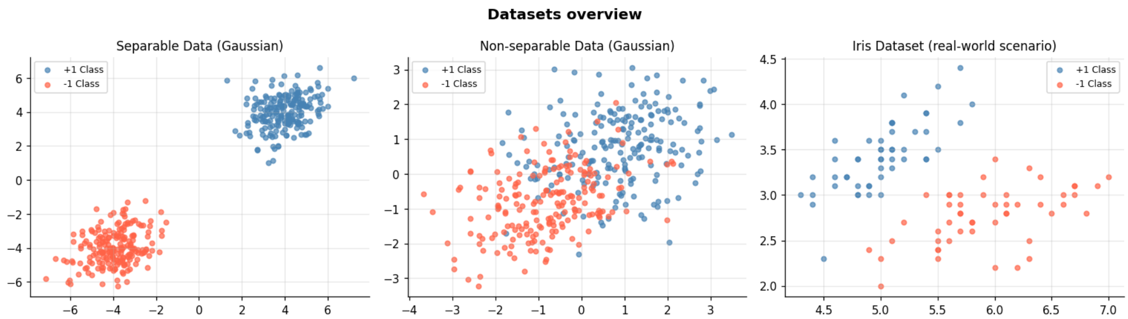

Figure 1: Graphical visualization of the datasets

2.3 Dataset Split and implementation

Splitting the dataset is a necessary condition in order to train a linear predictor and test it on different data, so every datasets pass through a split function that performs a random permutation of the samples and divides it in train and test sets based on a test set size. Real-world data may need to be scaled before utilization (like the Iris Dataset), so a scale based on mean and standard deviation of the raining set is performed. Only the training set mean and std are used in order to prevent any kind of data leakage in the test set, that could occur if we pre-scale all the dataset. Test set mean and std are not isolated and used to scale as better simulation of the real-world scenario when test data could not be previously available to be analyzed.

3 Implementation

3.1 Loss Functions

Hinge, Logistic and squared losses are implemented for the gradient step and also as aggregate loss functions over all the dataset for statistics. The gradient steps are calculated as simply the gradient of the loss functions, calculating the margin $y_t \cdot w^\top x_t$ and adding to that the regularization term $\nabla_w \left(\frac{\lambda}{2}\|w\|^2\right) = \lambda w$ excluding the bias. For better clarity the *-2* derived from the squared loss differentiation is explicitly maintained within the gradient function rather than being absorbed into the learning rate, so that the loss functions are the only thing changing in the training phase.

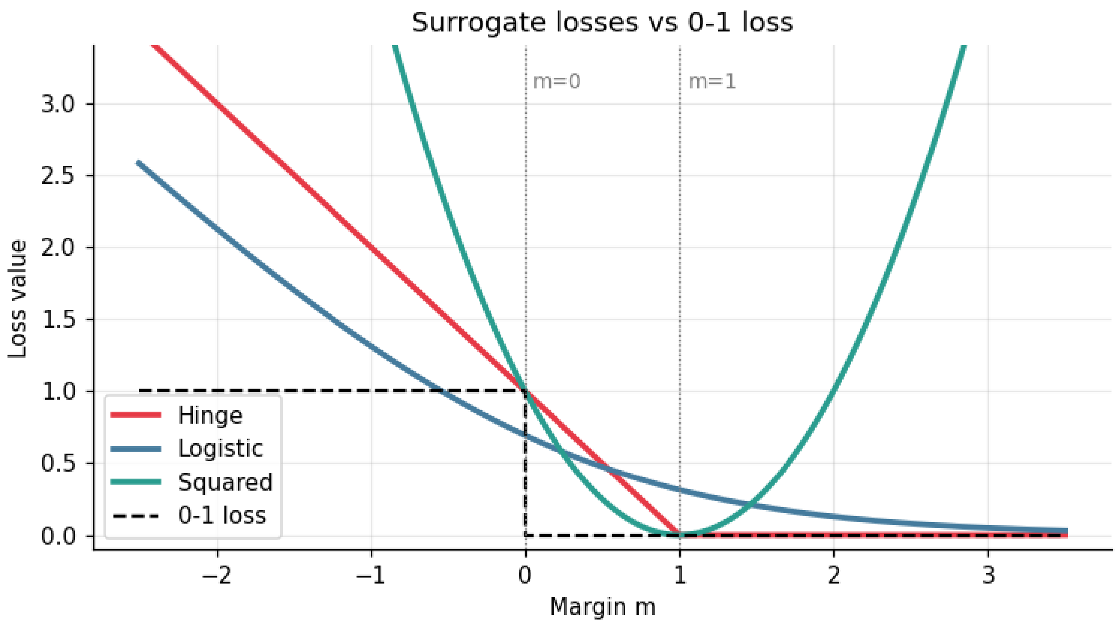

Figure 2: Loss Functions

3.2 Training

All classifiers are trained with the same SGD algorithm, as general as able to get in input different functions. In order to have an homogeneous hyperplane and to not have to deal with the bias, an extra feature is added to the data with value 1 and then the weight vector is initialized $w_1 = 0$ following the pseudo-code in subsection 1.3. In order to keep the algorithm as general as possible, the concept of strongly convex functions has not been applied to the algorithm, nor to the learning rate update, as it has been defined and kept as $\eta_t = \frac{\eta_0}{\sqrt{t}}$ At the end of each epoch the statistical data is collected and return together with the $\overline{w}$. In order to train all the linear classifiers and make them ready to be analyzed, a generic function to run them together and organize the results is the best way.

3.3 Plotting

All the plot utilities are defined as generic functions requiring the results produced in training and allowing a more structured and generic execution.

4 Experiments

For each type of Dataset, the experiments are focused on:

1. Showing the the test error, training error and the mean value of the surrogate loss (these two evaluated in the last epoch using the last values of w)

2. Plotting the relationship between Surrogate Loss and the training iterations (epochs)

3. Plotting the relationship between Classification Error and Surrogate Loss

The following experiments are run with always the same parameters that are $T = 2000, lam = 0.01, eta_0 = 0.05$ as number of rounds, regularization term and starting learning rate

4.1 Synthetic Separable Data

4.1.1 Measurements

Loss	Train Error	Test Error	Surrogate (last)
Hinge	0.000	0.000	0.0021
Logistic	0.000	0.000	0.0492
Squared	0.000	0.000	0.0420

Table 1: Performance on the separable dataset

The results on the separable dataset show that the training error reached 0, and the test error is also null. The surrogate loss values are instead small but not null, primary due to the presence of the regularization term for the hinge loss, the asymptotic nature of the logistic loss and the strong penalties of the squared loss.

4.1.2 Surrogate Loss vs Training Iterations

The loss functions all converges to nearly zero after some epochs, their nature is similar to what we expect from their mathematical properties, and what is in the measurements, with a strong descent initial part.

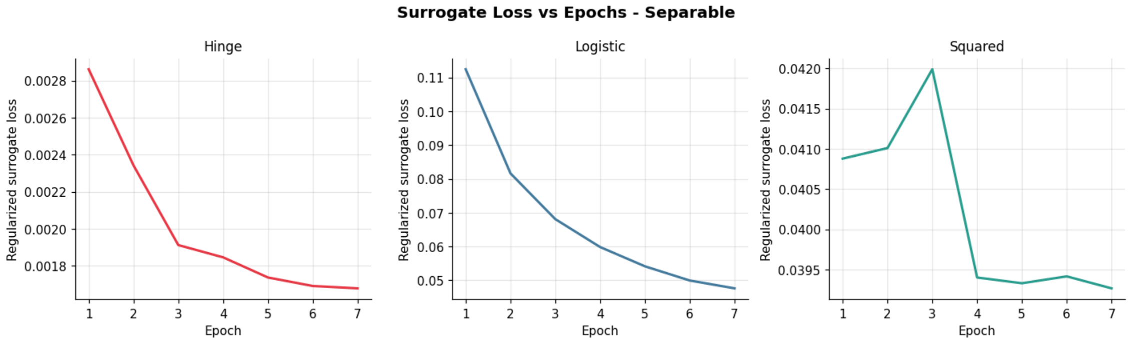

Figure 3: Surrogate Loss vs Training Iterations - Separable Data

4.1.3 Classification Error vs Surrogate Loss

In a clear case of separable data, the 0-1 error for all losses is equal to 0 even at the first epoch, and remains stable as the surrogate loss value decreases.

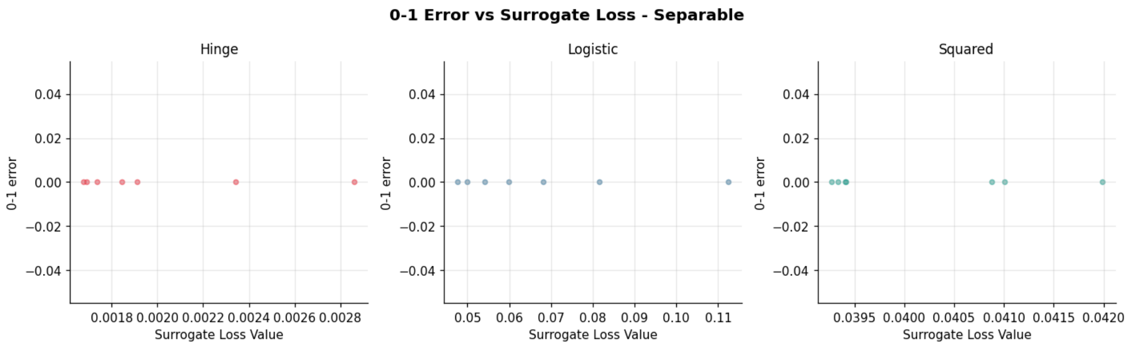

Figure 4: Classification Error vs Surrogate Loss - Separable Data

4.2 Synthetic Non-Separable Data

4.2.1 Measurements

Loss	Train Error	Test Error	Surrogate (last)
Hinge	0.177	0.180	0.4122
Logistic	0.173	0.180	0.3893
Squared	0.190	0.190	0.5227

Table 2: Performance on the non-separable dataset

The results on the non-separable dataset show that the training error doesn't reach 0 value, same as the test error, as expected with a linear predictor. Logistic and Hinge losses are similar, while squared loss is the worst one in terms of performance.

4.2.2 Surrogate Loss vs Training Iterations

When analyzing the surrogate loss over epochs in the non-separable setting, both the Hinge and Logistic losses exhibit a monotonic decay before reaching a stable asymptote. However, unlike the separable scenario, the surrogate losses do not converge toward zero. The squared loss presents the main problem of the quadratic penalty, struggling to stabilize.

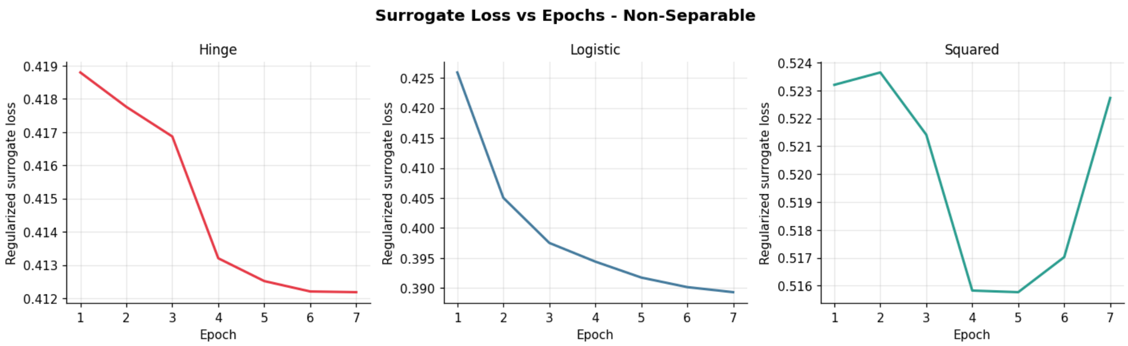

Figure 5: Surrogate Loss vs Training Iterations - Non Separable Data

4.2.3 Classification Error vs Surrogate Loss

In the non-separable case, the relationship between the surrogate loss and the 0-1 classification error highlights that while the surrogate losses decrease smoothly epoch after epoch, the 0-1 training error has strong fluctuations. This mismatch occurs because the 0-1 error is a discrete step function that changes when a data point physically crosses the decision boundary, while the surrogate losses are continuous and differentiable and they smoothly register even minor adjustments in the hyperplane's position and confidence margins.

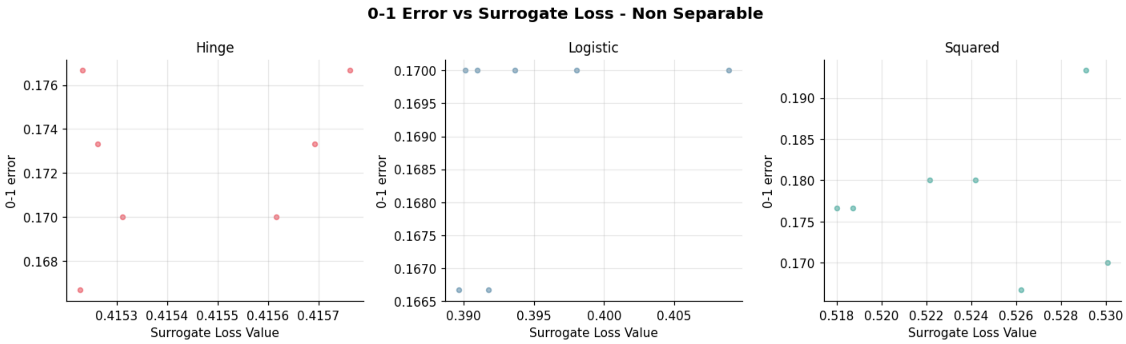

Figure 6: Classification Error vs Surrogate Loss - Non Separable Data

4.3 Real World Data

The Iris dataset has a smaller sample size, that means that maintaining the same number of rounds will result in more epochs, since the goal is to confront losses and not datasets and the real-world is the most interesting one, the final decision is to keep it this way instead of make the number of rounds dynamic to the sample size.

4.3.1 Measurements

Loss	Train Error	Test Error	Surrogate (last)
Hinge	0.013	0.000	0.2539
Logistic	0.027	0.000	0.4941
Squared	0.027	0.000	0.2105

Table 3: Performance comparison on the Iris dataset

Real world dataset (Iris) presents a smaller sample size and data linearly separable but with a smaller margin than the synthetic dataset, so we have many more epochs and the test error converges to zero, while we still have some training error and higher values of the surrogate losses.

4.3.2 Surrogate Loss vs Training Iterations

In the Iris dataset evaluation, the progression of the surrogate losses over training epochs highlights the structural contrast in how each mathematical formulation handles dense, compact feature clusters. While all three models achieve long-term minimization, their trajectories are not so smooth. The Hinge loss displays a highly adaptive path, while the Logistic loss struggle to deacy due to the high density of the Iris clusters. The Squared loss features the steepest initial gradient, remaining tightly locked in place despite late-stage gradient perturbations. Every loss presents spikes due to the data density and so the different samples picked up.

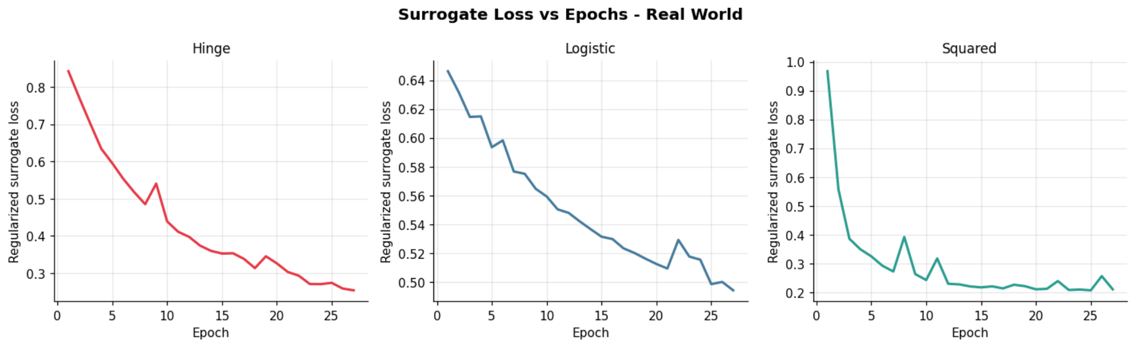

Figure 7: Surrogate Loss vs Training Iterations - Real World Data

4.3.3 Classification Error vs Surrogate Loss

The cross-analysis between the continuous surrogate losses and the discrete 0-1 training error uncovers the high spatial density of the Iris feature space. Throughout the training history, all models exhibit sudden 'slumps' or severe volatility spikes. The fact that every model ultimately converges to null test error demonstrates that these intense training-phase fluctuations are merely non-destructive exploratory steps, allowing the continuous surrogates to successfully locate an optimal, generalizing separating hyperplane.

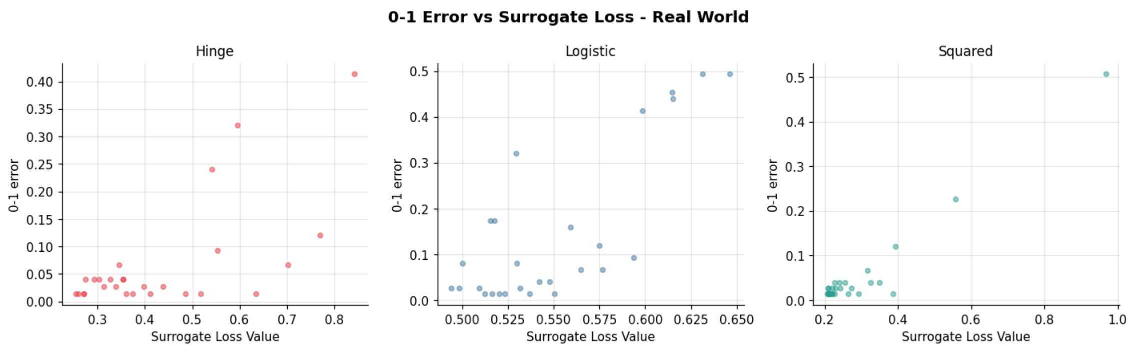

Figure 8: Classification Error vs Surrogate Loss - Real World Data

5 Extensions

5.1 Decision Boundaries

In order to show the decision boundaries of the models, the plot boundaries function is designed to visualize it on the plot and highlighting the regions created. To do so, a standard grid of 300x300 is created and each point is classified by the model, highlighting the line created and plotting over it the real points.

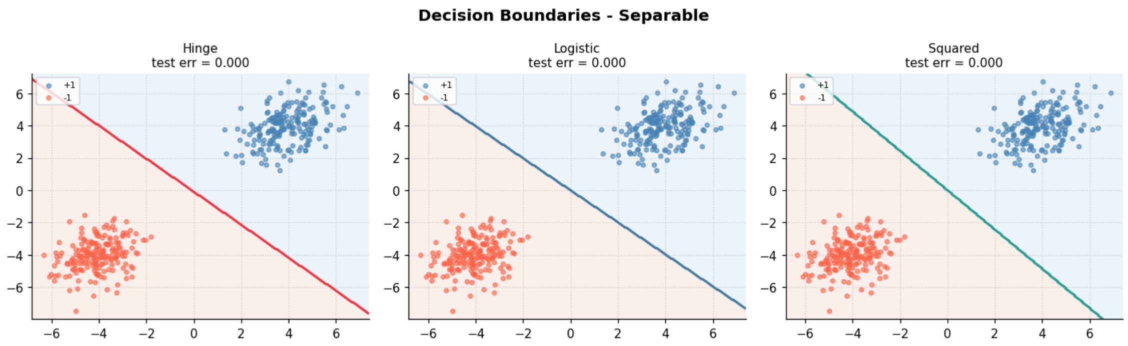

Figure 9: Decision Boundaries - Separable Dataset

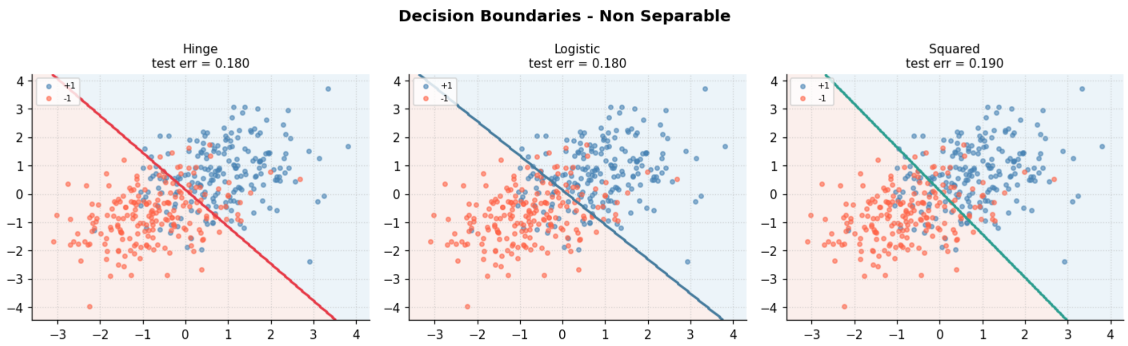

Figure 10: Decision Boundaries - Non Separable Dataset

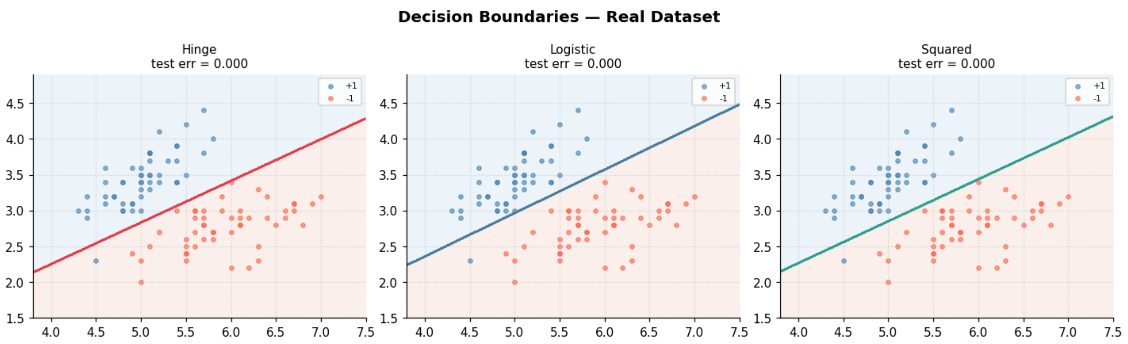

Figure 11: Decision Boundaries - Real World Dataset

5.2 Effect of regularization

In order to test the effect of regularization, separable data and real world data has been utilized. In both case the ideal value for λ seems to be in the middle, like 0.001.

Loss	$\boldsymbol{\lambda}$	**Train Err**	**Test Err**	**Surrogate**
Hinge	0.0	0.000	0.000	0.0007
Logistic	0.0	0.000	0.000	0.0102
Squared	0.0	0.000	0.000	0.0407
Hinge	0.0001	0.000	0.000	0.0001
Logistic	0.0001	0.000	0.000	0.0102
Squared	0.0001	0.000	0.000	0.0392
Hinge	0.001	0.000	0.000	0.0001
Logistic	0.001	0.000	0.000	0.0107
Squared	0.001	0.000	0.000	0.0390
Hinge	0.01	0.000	0.000	0.0009
Logistic	0.01	0.000	0.000	0.0148
Squared	0.01	0.000	0.000	0.0395
Hinge	0.1	0.000	0.000	0.0062
Logistic	0.1	0.000	0.000	0.0481
Squared	0.1	0.000	0.000	0.0407
Hinge	1.0	0.000	0.000	0.0395
Logistic	1.0	0.000	0.000	0.1853
Squared	1.0	0.000	0.000	0.0545

Table 4: Effect of regularization parameter λ on different surrogate losses on the Synthetic Separable Dataset

Loss	$\boldsymbol{\lambda}$	**Train Err**	**Test Err**	**Surrogate**
Hinge	0.0	0.000	0.040	0.1963
Logistic	0.0	0.080	0.080	0.4775
Squared	0.0	0.000	0.040	0.1684
Hinge	0.0001	0.013	0.040	0.2036
Logistic	0.0001	0.027	0.080	0.4706
Squared	0.0001	0.027	0.040	0.1717
Hinge	0.001	0.027	0.040	0.2032
Logistic	0.001	0.040	0.040	0.4706
Squared	0.001	0.000	0.040	0.1624
Hinge	0.01	0.000	0.040	0.2255
Logistic	0.01	0.000	0.080	0.4792
Squared	0.01	0.027	0.040	0.1826
Hinge	0.1	0.027	0.040	0.3766
Logistic	0.1	0.027	0.080	0.5399
Squared	0.1	0.013	0.040	0.2678
Hinge	1.0	0.453	0.640	0.8249
Logistic	1.0	0.413	0.360	0.6585
Squared	1.0	0.000	0.040	0.6717

Table 5: Effect of regularization parameter λ on different surrogate losses on Real World Dataset

5.3 Convergence Speed of Optimization Methods

The easier Optimization Method to implement in order to test convergence speed is the SGD with Momentum, that means adding velocity as momentum coefficient accumulating past gradients. The convergence speed is notably higher as shown in the plot testing on the Real World Dataset using the Logistic Loss.

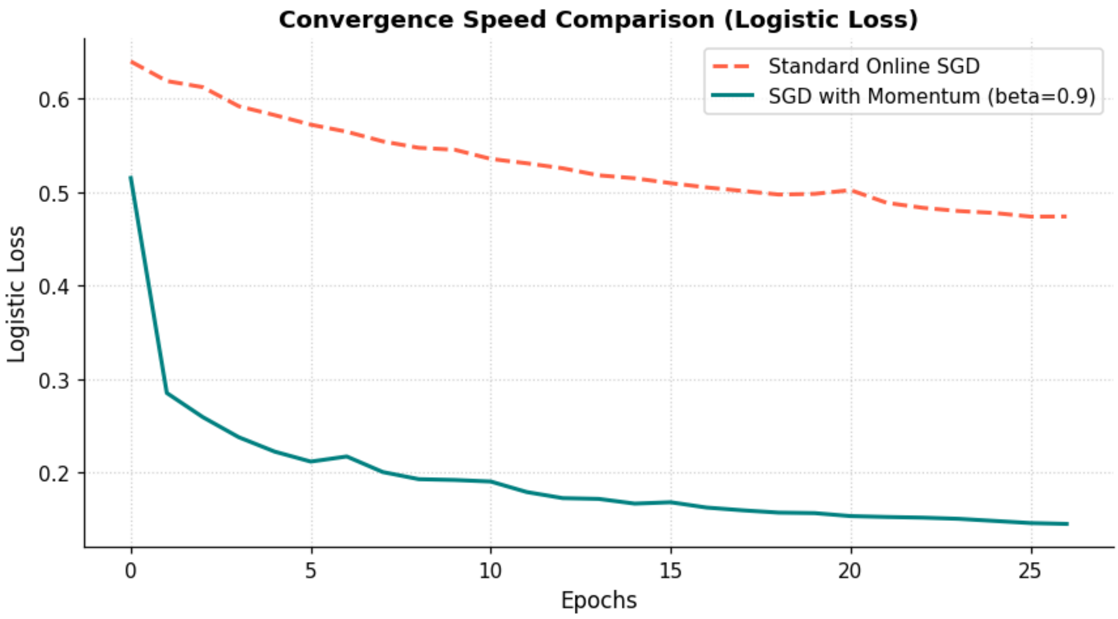

Figure 12: Convergence Speed - Real World Dataset - Logistic Loss

6 Concluding remarks

In conclusion, while these surrogate losses make the otherwise intractable 0-1 optimization computationally feasible, selecting the right surrogate depends heavily on the data distribution, noise levels, and the specific trade-offs between margin maximization and outlier sensitivity.

7 Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

References

[1] Jon D. McAuliffe Peter L. Bartlett Micheal I. Jorda. "Convexity, Classification, and Risk Bounds". In: *Technical Report 638* (November 4, 2003).